Event ends in 2 days 2 hours 54 minutes.

This implementation section brings together all Infrastructure phase specifications into a complete, production-ready AWS infrastructure for your chatbot application. You'll implement AWS resources, security controls, deployment pipelines, and monitoring systems.

Implementation Overview

Based on your Infrastructure specifications, you'll implement:

- **Complete AWS infrastructure** with serverless architecture and cost optimization
- **Comprehensive security controls** with defense-in-depth and compliance frameworks
- **Automated deployment pipeline** with CI/CD, quality gates, and rollback capabilities
- **Operational excellence** with monitoring, alerting, and incident response

Step 1: AWS CDK Infrastructure Setup

Implement the complete AWS infrastructure using CDK:

~~~
Create the AWS CDK infrastructure foundation:

```bash
# Initialize CDK project
mkdir infrastructure
cd infrastructure
npx aws-cdk init app --language typescript

# Install additional CDK dependencies
npm install @aws-cdk/aws-lambda @aws-cdk/aws-apigateway @aws-cdk/aws-dynamodb
npm install @aws-cdk/aws-cognito @aws-cdk/aws-s3 @aws-cdk/aws-cloudfront
npm install @aws-cdk/aws-wafv2 @aws-cdk/aws-guardduty @aws-cdk/aws-securityhub
npm install @aws-cdk/aws-cloudwatch @aws-cdk/aws-logs @aws-cdk/aws-sns
```

**Main Infrastructure Stack (lib/chatbot-infrastructure-stack.ts)**:
```typescript
import * as cdk from 'aws-cdk-lib';
import * as lambda from 'aws-cdk-lib/aws-lambda';
import * as apigateway from 'aws-cdk-lib/aws-apigateway';
import * as dynamodb from 'aws-cdk-lib/aws-dynamodb';
import * as cognito from 'aws-cdk-lib/aws-cognito';
import * as s3 from 'aws-cdk-lib/aws-s3';
import * as cloudfront from 'aws-cdk-lib/aws-cloudfront';
import * as wafv2 from 'aws-cdk-lib/aws-wafv2';
import * as iam from 'aws-cdk-lib/aws-iam';
import { Construct } from 'constructs';

export class ChatbotInfrastructureStack extends cdk.Stack {
  constructor(scope: Construct, id: string, props?: cdk.StackProps) {
    super(scope, id, props);

    // Create core infrastructure components
    this.createDatabaseLayer();
    this.createAuthenticationLayer();
    this.createComputeLayer();
    this.createApiLayer();
    this.createStorageLayer();
    this.createSecurityLayer();
    this.createMonitoringLayer();
  }
}
```
~~~

Step 2: Database and Storage Implementation

Implemen t comprehensive database and storage solutions:

~~~
Create database and storage infrastructure:

**Database Layer (lib/constructs/database-construct.ts)**:
```typescript
import * as cdk from 'aws-cdk-lib';
import * as dynamodb from 'aws-cdk-lib/aws-dynamodb';
import * as s3 from 'aws-cdk-lib/aws-s3';
import { Construct } from 'constructs';

export class DatabaseConstruct extends Construct {
  public readonly userTable: dynamodb.Table;
  public readonly conversationTable: dynamodb.Table;
  public readonly messageTable: dynamodb.Table;
  public readonly auditTable: dynamodb.Table;
  public readonly fileStorageBucket: s3.Bucket;

  constructor(scope: Construct, id: string) {
    super(scope, id);

    // User management table
    this.userTable = new dynamodb.Table(this, 'UserTable', {
      tableName: 'mycoolchatbot-users',
      partitionKey: { name: 'userId', type: dynamodb.AttributeType.STRING },
      billingMode: dynamodb.BillingMode.PAY_PER_REQUEST,
      encryption: dynamodb.TableEncryption.AWS_MANAGED,
      pointInTimeRecovery: true,
      removalPolicy: cdk.RemovalPolicy.RETAIN,
      stream: dynamodb.StreamViewType.NEW_AND_OLD_IMAGES
    });

    // Add GSI for email lookup
    this.userTable.addGlobalSecondaryIndex({
      indexName: 'EmailIndex',
      partitionKey: { name: 'email', type: dynamodb.AttributeType.STRING },
      projectionType: dynamodb.ProjectionType.ALL
    });

    // Conversation management table
    this.conversationTable = new dynamodb.Table(this, 'ConversationTable', {
      tableName: 'mycoolchatbot-conversations',
      partitionKey: { name: 'conversationId', type: dynamodb.AttributeType.STRING },
      sortKey: { name: 'userId', type: dynamodb.AttributeType.STRING },
      billingMode: dynamodb.BillingMode.PAY_PER_REQUEST,
      encryption: dynamodb.TableEncryption.AWS_MANAGED,
      pointInTimeRecovery: true,
      removalPolicy: cdk.RemovalPolicy.RETAIN
    });

    // Add GSI for user conversations
    this.conversationTable.addGlobalSecondaryIndex({
      indexName: 'UserConversationsIndex',
      partitionKey: { name: 'userId', type: dynamodb.AttributeType.STRING },
      sortKey: { name: 'updatedAt', type: dynamodb.AttributeType.STRING },
      projectionType: dynamodb.ProjectionType.ALL
    });

    // Message storage table
    this.messageTable = new dynamodb.Table(this, 'MessageTable', {
      tableName: 'mycoolchatbot-messages',
      partitionKey: { name: 'conversationId', type: dynamodb.AttributeType.STRING },
      sortKey: { name: 'messageId', type: dynamodb.AttributeType.STRING },
      billingMode: dynamodb.BillingMode.PAY_PER_REQUEST,
      encryption: dynamodb.TableEncryption.AWS_MANAGED,
      pointInTimeRecovery: true,
      removalPolicy: cdk.RemovalPolicy.RETAIN
    });

    // Add GSI for timestamp-based queries
    this.messageTable.addGlobalSecondaryIndex({
      indexName: 'TimestampIndex',
      partitionKey: { name: 'conversationId', type: dynamodb.AttributeType.STRING },
      sortKey: { name: 'timestamp', type: dynamodb.AttributeType.STRING },
      projectionType: dynamodb.ProjectionType.ALL
    });

    // Audit logging table
    this.auditTable = new dynamodb.Table(this, 'AuditTable', {
      tableName: 'mycoolchatbot-audit-logs',
      partitionKey: { name: 'eventId', type: dynamodb.AttributeType.STRING },
      sortKey: { name: 'timestamp', type: dynamodb.AttributeType.STRING },
      billingMode: dynamodb.BillingMode.PAY_PER_REQUEST,
      encryption: dynamodb.TableEncryption.AWS_MANAGED,
      pointInTimeRecovery: true,
      removalPolicy: cdk.RemovalPolicy.RETAIN,
      timeToLiveAttribute: 'ttl'
    });

    // File storage bucket
    this.fileStorageBucket = new s3.Bucket(this, 'FileStorageBucket', {
      bucketName: `mycoolchatbot-files-${cdk.Aws.ACCOUNT_ID}-${cdk.Aws.REGION}`,
      encryption: s3.BucketEncryption.S3_MANAGED,
      blockPublicAccess: s3.BlockPublicAccess.BLOCK_ALL,
      versioned: true,
      lifecycleRules: [{
        id: 'DeleteOldVersions',
        noncurrentVersionExpiration: cdk.Duration.days(30)
      }],
      removalPolicy: cdk.RemovalPolicy.RETAIN
    });
  }
}
```

**Caching Layer (lib/constructs/cache-construct.ts)**:
```typescript
import * as cdk from 'aws-cdk-lib';
import * as elasticache from 'aws-cdk-lib/aws-elasticache';
import * as ec2 from 'aws-cdk-lib/aws-ec2';
import { Construct } from 'constructs';

export class CacheConstruct extends Construct {
  public readonly redisCluster: elasticache.CfnReplicationGroup;
  public readonly cacheSecurityGroup: ec2.SecurityGroup;

  constructor(scope: Construct, id: string, vpc: ec2.Vpc) {
    super(scope, id);

    // Create cache subnet group
    const cacheSubnetGroup = new elasticache.CfnSubnetGroup(this, 'CacheSubnetGroup', {
      description: 'Subnet group for Redis cache',
      subnetIds: vpc.privateSubnets.map(subnet => subnet.subnetId),
      cacheSubnetGroupName: 'mycoolchatbot-cache-subnet-group'
    });

    // Create security group for cache
    this.cacheSecurityGroup = new ec2.SecurityGroup(this, 'CacheSecurityGroup', {
      vpc,
      description: 'Security group for Redis cache',
      allowAllOutbound: false
    });

    // Allow Redis access from Lambda functions
    this.cacheSecurityGroup.addIngressRule(
      ec2.Peer.ipv4(vpc.vpcCidrBlock),
      ec2.Port.tcp(6379),
      'Allow Redis access from VPC'
    );

    // Create Redis replication group
    this.redisCluster = new elasticache.CfnReplicationGroup(this, 'RedisCluster', {
      replicationGroupDescription: 'Redis cluster for MyCoolChatbot caching',
      numCacheClusters: 2,
      engine: 'redis',
      cacheNodeType: 'cache.t3.micro',
      port: 6379,
      cacheSubnetGroupName: cacheSubnetGroup.cacheSubnetGroupName,
      securityGroupIds: [this.cacheSecurityGroup.securityGroupId],
      atRestEncryptionEnabled: true,
      transitEncryptionEnabled: true,
      multiAzEnabled: true,
      automaticFailoverEnabled: true,
      engineVersion: '7.0'
    });
  }
}
```
~~~

Step 3: Authentication and Security ImplementationI

mplement comprehensive authentication and security infrastructure:

~~~
Create authentication and security layers:

Authentication Layer (lib/constructs/auth-construct.ts):
```typescript
import * as cdk from 'aws-cdk-lib';
import * as cognito from 'aws-cdk-lib/aws-cognito';
import * as iam from 'aws-cdk-lib/aws-iam';
import { Construct } from 'constructs';

export class AuthConstruct extends Construct {
  public readonly userPool: cognito.UserPool;
  public readonly userPoolClient: cognito.UserPoolClient;
  public readonly identityPool: cognito.CfnIdentityPool;

  constructor(scope: Construct, id: string) {
    super(scope, id);

    // Create Cognito User Pool
    this.userPool = new cognito.UserPool(this, 'UserPool', {
      userPoolName: 'mycoolchatbot-user-pool',
      selfSignUpEnabled: true,
      signInAliases: {
        email: true,
        username: false
      },
      autoVerify: {
        email: true
      },
      standardAttributes: {
        email: {
          required: true,
          mutable: true
        },
        givenName: {
          required: true,
          mutable: true
        },
        familyName: {
          required: true,
          mutable: true
        }
      },
      passwordPolicy: {
        minLength: 12,
        requireLowercase: true,
        requireUppercase: true,
        requireDigits: true,
        requireSymbols: true
      },
      mfa: cognito.Mfa.OPTIONAL,
      mfaSecondFactor: {
        sms: true,
        otp: true
      },
      accountRecovery: cognito.AccountRecovery.EMAIL_ONLY,
      removalPolicy: cdk.RemovalPolicy.RETAIN
    });

    // Add MFA configuration
    this.userPool.addDomain('UserPoolDomain', {
      cognitoDomain: {
        domainPrefix: `mycoolchatbot-auth-${cdk.Aws.ACCOUNT_ID}`
      }
    });

    // Create User Pool Client
    this.userPoolClient = new cognito.UserPoolClient(this, 'UserPoolClient', {
      userPool: this.userPool,
      userPoolClientName: 'mycoolchatbot-web-client',
      generateSecret: false,
      authFlows: {
        userPassword: true,
        userSrp: true,
        custom: true,
        adminUserPassword: false
      },
      oAuth: {
        flows: {
          authorizationCodeGrant: true,
          implicitCodeGrant: false
        },
        scopes: [
          cognito.OAuthScope.EMAIL,
          cognito.OAuthScope.OPENID,
          cognito.OAuthScope.PROFILE
        ],
        callbackUrls: ['http://localhost:3000/auth/callback'],
        logoutUrls: ['http://localhost:3000/auth/logout']
      },
      preventUserExistenceErrors: true,
      refreshTokenValidity: cdk.Duration.days(30),
      accessTokenValidity: cdk.Duration.minutes(60),
      idTokenValidity: cdk.Duration.minutes(60)
    });

    // Create Identity Pool
    this.identityPool = new cognito.CfnIdentityPool(this, 'IdentityPool', {
      identityPoolName: 'mycoolchatbot-identity-pool',
      allowUnauthenticatedIdentities: false,
      cognitoIdentityProviders: [{
        clientId: this.userPoolClient.userPoolClientId,
        providerName: this.userPool.userPoolProviderName,
        serverSideTokenCheck: true
      }]
    });

    // Create IAM roles for authenticated users
    const authenticatedRole = new iam.Role(this, 'AuthenticatedRole', {
      assumedBy: new iam.FederatedPrincipal(
        'cognito-identity.amazonaws.com',
        {
          StringEquals: {
            'cognito-identity.amazonaws.com:aud': this.identityPool.ref
          },
          'ForAnyValue:StringLike': {
            'cognito-identity.amazonaws.com:amr': 'authenticated'
          }
        },
        'sts:AssumeRoleWithWebIdentity'
      ),
      managedPolicies: [
        iam.ManagedPolicy.fromAwsManagedPolicyName('AmazonCognitoPowerUser')
      ]
    });

    // Attach role to identity pool
    new cognito.CfnIdentityPoolRoleAttachment(this, 'IdentityPoolRoleAttachment', {
      identityPoolId: this.identityPool.ref,
      roles: {
        authenticated: authenticatedRole.roleArn
      }
    });
  }
}
```

Security Layer (lib/constructs/security-construct.ts):
```typescript
import * as cdk from 'aws-cdk-lib';
import * as wafv2 from 'aws-cdk-lib/aws-wafv2';
import * as guardduty from 'aws-cdk-lib/aws-guardduty';
import * as securityhub from 'aws-cdk-lib/aws-securityhub';
import * as config from 'aws-cdk-lib/aws-config';
import * as kms from 'aws-cdk-lib/aws-kms';
import { Construct } from 'constructs';

export class SecurityConstruct extends Construct {
  public readonly webAcl: wafv2.CfnWebACL;
  public readonly kmsKey: kms.Key;

  constructor(scope: Construct, id: string) {
    super(scope, id);

    // Create KMS key for encryption
    this.kmsKey = new kms.Key(this, 'ChatbotKMSKey', {
      description: 'KMS key for MyCoolChatbot-KiroSpec application encryption',
      enableKeyRotation: true,
      removalPolicy: cdk.RemovalPolicy.RETAIN
    });

    // Create WAF Web ACL
    this.webAcl = new wafv2.CfnWebACL(this, 'WebACL', {
      name: 'mycoolchatbot-web-acl',
      scope: 'CLOUDFRONT',
      defaultAction: { allow: {} },
      description: 'Web ACL for MyCoolChatbot-KiroSpec application',
      rules: [
        {
          name: 'AWSManagedRulesCommonRuleSet',
          priority: 1,
          overrideAction: { none: {} },
          statement: {
            managedRuleGroupStatement: {
              vendorName: 'AWS',
              name: 'AWSManagedRulesCommonRuleSet'
            }
          },
          visibilityConfig: {
            sampledRequestsEnabled: true,
            cloudWatchMetricsEnabled: true,
            metricName: 'CommonRuleSetMetric'
          }
        },
        {
          name: 'AWSManagedRulesKnownBadInputsRuleSet',
          priority: 2,
          overrideAction: { none: {} },
          statement: {
            managedRuleGroupStatement: {
              vendorName: 'AWS',
              name: 'AWSManagedRulesKnownBadInputsRuleSet'
            }
          },
          visibilityConfig: {
            sampledRequestsEnabled: true,
            cloudWatchMetricsEnabled: true,
            metricName: 'KnownBadInputsMetric'
          }
        },
        {
          name: 'RateLimitRule',
          priority: 3,
          action: { block: {} },
          statement: {
            rateBasedStatement: {
              limit: 2000,
              aggregateKeyType: 'IP'
            }
          },
          visibilityConfig: {
            sampledRequestsEnabled: true,
            cloudWatchMetricsEnabled: true,
            metricName: 'RateLimitMetric'
          }
        }
      ],
      visibilityConfig: {
        sampledRequestsEnabled: true,
        cloudWatchMetricsEnabled: true,
        metricName: 'MyCoolChatbotWebACL'
      }
    });

    // Enable GuardDuty
    new guardduty.CfnDetector(this, 'GuardDutyDetector', {
      enable: true,
      findingPublishingFrequency: 'FIFTEEN_MINUTES'
    });

    // Enable Security Hub
    new securityhub.CfnHub(this, 'SecurityHub', {
      tags: [{
        key: 'Application',
        value: 'Chatbot'
      }]
    });

    // Enable Config
    const configRole = new cdk.aws_iam.Role(this, 'ConfigRole', {
      assumedBy: new cdk.aws_iam.ServicePrincipal('config.amazonaws.com'),
      managedPolicies: [
        cdk.aws_iam.ManagedPolicy.fromAwsManagedPolicyName('service-role/ConfigRole')
      ]
    });

    new config.CfnConfigurationRecorder(this, 'ConfigRecorder', {
      name: 'mycoolchatbot-config-recorder',
      roleArn: configRole.roleArn,
      recordingGroup: {
        allSupported: true,
        includeGlobalResourceTypes: true
      }
    });
  }
}
```
~~~

Step 4: Compute and API ImplementationImpleme

nt serverless compute and API infrastructure:

~~~
Create compute and API layers:

Compute Layer (lib/constructs/compute-construct.ts):
```typescript
import * as cdk from 'aws-cdk-lib';
import * as lambda from 'aws-cdk-lib/aws-lambda';
import * as nodejs from 'aws-cdk-lib/aws-lambda-nodejs';
import * as iam from 'aws-cdk-lib/aws-iam';
import * as ec2 from 'aws-cdk-lib/aws-ec2';
import { Construct } from 'constructs';

export class ComputeConstruct extends Construct {
  public readonly authFunction: nodejs.NodejsFunction;
  public readonly chatFunction: nodejs.NodejsFunction;
  public readonly userFunction: nodejs.NodejsFunction;
  public readonly aiFunction: nodejs.NodejsFunction;

  constructor(
    scope: Construct,
    id: string,
    vpc: ec2.Vpc,
    securityGroup: ec2.SecurityGroup
  ) {
    super(scope, id);

    // Common Lambda configuration
    const commonLambdaProps = {
      runtime: lambda.Runtime.NODEJS_18_X,
      timeout: cdk.Duration.seconds(30),
      memorySize: 512,
      vpc,
      securityGroups: [securityGroup],
      environment: {
        NODE_ENV: 'production',
        LOG_LEVEL: 'info'
      },
      bundling: {
        minify: true,
        sourceMap: true,
        target: 'es2020'
      }
    };

    // Authentication Lambda
    this.authFunction = new nodejs.NodejsFunction(this, 'AuthFunction', {
      ...commonLambdaProps,
      functionName: 'mycoolchatbot-auth-function',
      entry: 'src/lambda/auth/index.ts',
      description: 'Handles user authentication and authorization',
      environment: {
        ...commonLambdaProps.environment,
        USER_POOL_ID: process.env.USER_POOL_ID || '',
        USER_POOL_CLIENT_ID: process.env.USER_POOL_CLIENT_ID || ''
      }
    });

    // Chat management Lambda
    this.chatFunction = new nodejs.NodejsFunction(this, 'ChatFunction', {
      ...commonLambdaProps,
      functionName: 'mycoolchatbot-chat-function',
      entry: 'src/lambda/chat/index.ts',
      description: 'Handles chat operations and message management',
      timeout: cdk.Duration.seconds(60),
      memorySize: 1024
    });

    // User management Lambda
    this.userFunction = new nodejs.NodejsFunction(this, 'UserFunction', {
      ...commonLambdaProps,
      functionName: 'mycoolchatbot-user-function',
      entry: 'src/lambda/user/index.ts',
      description: 'Handles user profile and preferences management'
    });

    // AI integration Lambda
    this.aiFunction = new nodejs.NodejsFunction(this, 'AIFunction', {
      ...commonLambdaProps,
      functionName: 'mycoolchatbot-ai-function',
      entry: 'src/lambda/ai/index.ts',
      description: 'Handles AI model integration and response generation',
      timeout: cdk.Duration.seconds(120),
      memorySize: 2048,
      environment: {
        ...commonLambdaProps.environment,
        BEDROCK_REGION: cdk.Aws.REGION,
        MODEL_ID: 'anthropic.claude-3-sonnet-20240229-v1:0'
      }
    });

    // Grant Bedrock permissions to AI function
    this.aiFunction.addToRolePolicy(new iam.PolicyStatement({
      effect: iam.Effect.ALLOW,
      actions: [
        'bedrock:InvokeModel',
        'bedrock:InvokeModelWithResponseStream'
      ],
      resources: ['*']
    }));

    // Add CloudWatch Logs permissions
    [this.authFunction, this.chatFunction, this.userFunction, this.aiFunction].forEach(fn => {
      fn.addToRolePolicy(new iam.PolicyStatement({
        effect: iam.Effect.ALLOW,
        actions: [
          'logs:CreateLogGroup',
          'logs:CreateLogStream',
          'logs:PutLogEvents'
        ],
        resources: ['*']
      }));
    });
  }
}
```

API Gateway Layer (lib/constructs/api-construct.ts):
```typescript
import * as cdk from 'aws-cdk-lib';
import * as apigateway from 'aws-cdk-lib/aws-apigateway';
import * as lambda from 'aws-cdk-lib/aws-lambda';
import * as cognito from 'aws-cdk-lib/aws-cognito';
import * as wafv2 from 'aws-cdk-lib/aws-wafv2';
import { Construct } from 'constructs';

export class ApiConstruct extends Construct {
  public readonly api: apigateway.RestApi;
  public readonly authorizer: apigateway.CognitoUserPoolsAuthorizer;

  constructor(
    scope: Construct,
    id: string,
    userPool: cognito.UserPool,
    functions: {
      authFunction: lambda.Function;
      chatFunction: lambda.Function;
      userFunction: lambda.Function;
      aiFunction: lambda.Function;
    },
    webAcl: wafv2.CfnWebACL
  ) {
    super(scope, id);

    // Create API Gateway
    this.api = new apigateway.RestApi(this, 'ChatbotApi', {
      restApiName: 'mycoolchatbot-api',
      description: 'API for MyCoolChatbot-KiroSpec application',
      defaultCorsPreflightOptions: {
        allowOrigins: apigateway.Cors.ALL_ORIGINS,
        allowMethods: apigateway.Cors.ALL_METHODS,
        allowHeaders: [
          'Content-Type',
          'X-Amz-Date',
          'Authorization',
          'X-Api-Key',
          'X-Amz-Security-Token'
        ]
      },
      deployOptions: {
        stageName: 'prod',
        throttlingRateLimit: 1000,
        throttlingBurstLimit: 2000,
        loggingLevel: apigateway.MethodLoggingLevel.INFO,
        dataTraceEnabled: true,
        metricsEnabled: true
      }
    });

    // Create Cognito authorizer
    this.authorizer = new apigateway.CognitoUserPoolsAuthorizer(this, 'ApiAuthorizer', {
      cognitoUserPools: [userPool],
      authorizerName: 'mycoolchatbot-authorizer',
      identitySource: 'method.request.header.Authorization'
    });

    // Create API resources and methods
    this.createAuthRoutes(functions.authFunction);
    this.createUserRoutes(functions.userFunction);
    this.createChatRoutes(functions.chatFunction);
    this.createAiRoutes(functions.aiFunction);

    // Associate WAF with API Gateway
    new wafv2.CfnWebACLAssociation(this, 'ApiWafAssociation', {
      resourceArn: this.api.deploymentStage.stageArn,
      webAclArn: webAcl.attrArn
    });
  }

  private createAuthRoutes(authFunction: lambda.Function) {
    const auth = this.api.root.addResource('auth');

    // Public authentication endpoints
    auth.addResource('login').addMethod('POST',
      new apigateway.LambdaIntegration(authFunction)
    );

    auth.addResource('register').addMethod('POST',
      new apigateway.LambdaIntegration(authFunction)
    );

    auth.addResource('refresh').addMethod('POST',
      new apigateway.LambdaIntegration(authFunction)
    );

    // Protected authentication endpoints
    auth.addResource('logout').addMethod('POST',
      new apigateway.LambdaIntegration(authFunction),
      { authorizer: this.authorizer }
    );

    auth.addResource('profile').addMethod('GET',
      new apigateway.LambdaIntegration(authFunction),
      { authorizer: this.authorizer }
    );
  }

  private createUserRoutes(userFunction: lambda.Function) {
    const users = this.api.root.addResource('users');

    users.addMethod('GET',
      new apigateway.LambdaIntegration(userFunction),
      { authorizer: this.authorizer }
    );

    users.addMethod('PUT',
      new apigateway.LambdaIntegration(userFunction),
      { authorizer: this.authorizer }
    );

    const userById = users.addResource('{userId}');
    userById.addMethod('GET',
      new apigateway.LambdaIntegration(userFunction),
      { authorizer: this.authorizer }
    );
  }

  private createChatRoutes(chatFunction: lambda.Function) {
    const conversations = this.api.root.addResource('conversations');

    conversations.addMethod('GET',
      new apigateway.LambdaIntegration(chatFunction),
      { authorizer: this.authorizer }
    );

    conversations.addMethod('POST',
      new apigateway.LambdaIntegration(chatFunction),
      { authorizer: this.authorizer }
    );

    const conversationById = conversations.addResource('{conversationId}');
    conversationById.addMethod('GET',
      new apigateway.LambdaIntegration(chatFunction),
      { authorizer: this.authorizer }
    );

    conversationById.addMethod('DELETE',
      new apigateway.LambdaIntegration(chatFunction),
      { authorizer: this.authorizer }
    );

    const messages = conversationById.addResource('messages');
    messages.addMethod('GET',
      new apigateway.LambdaIntegration(chatFunction),
      { authorizer: this.authorizer }
    );

    messages.addMethod('POST',
      new apigateway.LambdaIntegration(chatFunction),
      { authorizer: this.authorizer }
    );
  }

  private createAiRoutes(aiFunction: lambda.Function) {
    const ai = this.api.root.addResource('ai');

    ai.addResource('chat').addMethod('POST',
      new apigateway.LambdaIntegration(aiFunction, {
        timeout: cdk.Duration.seconds(120)
      }),
      { authorizer: this.authorizer }
    );

    ai.addResource('stream').addMethod('POST',
      new apigateway.LambdaIntegration(aiFunction, {
        timeout: cdk.Duration.seconds(120)
      }),
      { authorizer: this.authorizer }
    );
  }
}
```
~~~

Step 5: Monitoring and Deployment ImplementationImpleme

nt comprehensive monitoring and deployment infrastructure:

~~~
Create monitoring and deployment systems:

Monitoring Layer (lib/constructs/monitoring-construct.ts):
```typescript
import * as cdk from 'aws-cdk-lib';
import * as cloudwatch from 'aws-cdk-lib/aws-cloudwatch';
import * as sns from 'aws-cdk-lib/aws-sns';
import * as snsSubscriptions from 'aws-cdk-lib/aws-sns-subscriptions';
import * as lambda from 'aws-cdk-lib/aws-lambda';
import * as apigateway from 'aws-cdk-lib/aws-apigateway';
import { Construct } from 'constructs';

export class MonitoringConstruct extends Construct {
  public readonly dashboard: cloudwatch.Dashboard;
  public readonly alertTopic: sns.Topic;

  constructor(
    scope: Construct,
    id: string,
    api: apigateway.RestApi,
    functions: lambda.Function[]
  ) {
    super(scope, id);

    // Create SNS topic for alerts
    this.alertTopic = new sns.Topic(this, 'AlertTopic', {
      topicName: 'mycoolchatbot-alerts',
      displayName: 'MyCoolChatbot-KiroSpec Application Alerts'
    });

    // Add email subscription (replace with actual email)
    this.alertTopic.addSubscription(
      new snsSubscriptions.EmailSubscription('admin@example.com')
    );

    // Create CloudWatch Dashboard
    this.dashboard = new cloudwatch.Dashboard(this, 'ChatbotDashboard', {
      dashboardName: 'mycoolchatbot-application-dashboard'
    });

    // Add API Gateway metrics
    this.addApiMetrics(api);

    // Add Lambda metrics
    this.addLambdaMetrics(functions);

    // Add custom business metrics
    this.addBusinessMetrics();

    // Create alarms
    this.createAlarms(api, functions);
  }

  private addApiMetrics(api: apigateway.RestApi) {
    const apiMetrics = [
      new cloudwatch.Metric({
        namespace: 'AWS/ApiGateway',
        metricName: 'Count',
        dimensionsMap: { ApiName: api.restApiName },
        statistic: 'Sum'
      }),
      new cloudwatch.Metric({
        namespace: 'AWS/ApiGateway',
        metricName: 'Latency',
        dimensionsMap: { ApiName: api.restApiName },
        statistic: 'Average'
      }),
      new cloudwatch.Metric({
        namespace: 'AWS/ApiGateway',
        metricName: '4XXError',
        dimensionsMap: { ApiName: api.restApiName },
        statistic: 'Sum'
      }),
      new cloudwatch.Metric({
        namespace: 'AWS/ApiGateway',
        metricName: '5XXError',
        dimensionsMap: { ApiName: api.restApiName },
        statistic: 'Sum'
      })
    ];

    this.dashboard.addWidgets(
      new cloudwatch.GraphWidget({
        title: 'API Gateway Metrics',
        left: apiMetrics,
        width: 12,
        height: 6
      })
    );
  }

  private addLambdaMetrics(functions: lambda.Function[]) {
    functions.forEach((fn, index) => {
      const lambdaMetrics = [
        new cloudwatch.Metric({
          namespace: 'AWS/Lambda',
          metricName: 'Invocations',
          dimensionsMap: { FunctionName: fn.functionName },
          statistic: 'Sum'
        }),
        new cloudwatch.Metric({
          namespace: 'AWS/Lambda',
          metricName: 'Duration',
          dimensionsMap: { FunctionName: fn.functionName },
          statistic: 'Average'
        }),
        new cloudwatch.Metric({
          namespace: 'AWS/Lambda',
          metricName: 'Errors',
          dimensionsMap: { FunctionName: fn.functionName },
          statistic: 'Sum'
        }),
        new cloudwatch.Metric({
          namespace: 'AWS/Lambda',
          metricName: 'Throttles',
          dimensionsMap: { FunctionName: fn.functionName },
          statistic: 'Sum'
        })
      ];

      this.dashboard.addWidgets(
        new cloudwatch.GraphWidget({
          title: `Lambda Metrics - ${fn.functionName}`,
          left: lambdaMetrics,
          width: 6,
          height: 6
        })
      );
    });
  }

  private addBusinessMetrics() {
    // Custom metrics for business KPIs
    const businessMetrics = [
      new cloudwatch.Metric({
        namespace: 'Chatbot/Business',
        metricName: 'ActiveUsers',
        statistic: 'Sum'
      }),
      new cloudwatch.Metric({
        namespace: 'Chatbot/Business',
        metricName: 'ConversationsStarted',
        statistic: 'Sum'
      }),
      new cloudwatch.Metric({
        namespace: 'Chatbot/Business',
        metricName: 'MessagesProcessed',
        statistic: 'Sum'
      }),
      new cloudwatch.Metric({
        namespace: 'Chatbot/Business',
        metricName: 'AIResponseTime',
        statistic: 'Average'
      })
    ];

    this.dashboard.addWidgets(
      new cloudwatch.GraphWidget({
        title: 'Business Metrics',
        left: businessMetrics,
        width: 12,
        height: 6
      })
    );
  }

  private createAlarms(api: apigateway.RestApi, functions: lambda.Function[]) {
    // API Gateway error rate alarm
    const apiErrorAlarm = new cloudwatch.Alarm(this, 'ApiErrorAlarm', {
      alarmName: 'mycoolchatbot-api-error-rate',
      alarmDescription: 'API Gateway error rate is too high',
      metric: new cloudwatch.Metric({
        namespace: 'AWS/ApiGateway',
        metricName: '5XXError',
        dimensionsMap: { ApiName: api.restApiName },
        statistic: 'Sum'
      }),
      threshold: 10,
      evaluationPeriods: 2,
      comparisonOperator: cloudwatch.ComparisonOperator.GREATER_THAN_THRESHOLD
    });

    apiErrorAlarm.addAlarmAction(
      new cdk.aws_cloudwatch_actions.SnsAction(this.alertTopic)
    );

    // Lambda error rate alarms
    functions.forEach(fn => {
      const lambdaErrorAlarm = new cloudwatch.Alarm(this, `${fn.functionName}ErrorAlarm`, {
        alarmName: `${fn.functionName}-error-rate`,
        alarmDescription: `${fn.functionName} error rate is too high`,
        metric: new cloudwatch.Metric({
          namespace: 'AWS/Lambda',
          metricName: 'Errors',
          dimensionsMap: { FunctionName: fn.functionName },
          statistic: 'Sum'
        }),
        threshold: 5,
        evaluationPeriods: 2,
        comparisonOperator: cloudwatch.ComparisonOperator.GREATER_THAN_THRESHOLD
      });

      lambdaErrorAlarm.addAlarmAction(
        new cdk.aws_cloudwatch_actions.SnsAction(this.alertTopic)
      );
    });
  }
}
```

Deployment Pipeline (scripts/deploy.sh):
```bash
#!/bin/bash

set -e

# Configuration
ENVIRONMENT=${1:-dev}
REGION=${2:-us-east-1}
PROFILE=${3:-default}

echo "🚀 Starting deployment for environment: $ENVIRONMENT"

# Validate environment
if [[ ! "$ENVIRONMENT" =~ ^(dev|staging|prod)$ ]]; then
    echo "❌ Invalid environment. Use: dev, staging, or prod"
    exit 1
fi

# Set environment-specific variables
case $ENVIRONMENT in
    "dev")
        STACK_NAME="mycoolchatbot-dev"
        DOMAIN_NAME="dev.mycoolchatbot.example.com"
        ;;
    "staging")
        STACK_NAME="mycoolchatbot-staging"
        DOMAIN_NAME="staging.mycoolchatbot.example.com"
        ;;
    "prod")
        STACK_NAME="mycoolchatbot-prod"
        DOMAIN_NAME="mycoolchatbot.example.com"
        ;;
esac

echo "📋 Deployment Configuration:"
echo "  Environment: $ENVIRONMENT"
echo "  Stack Name: $STACK_NAME"
echo "  Region: $REGION"
echo "  Domain: $DOMAIN_NAME"

# Pre-deployment checks
echo "🔍 Running pre-deployment checks..."

# Check AWS CLI configuration
if ! aws sts get-caller-identity --profile $PROFILE > /dev/null 2>&1; then
    echo "❌ AWS CLI not configured properly"
    exit 1
fi

# Check CDK CLI
if ! command -v cdk &> /dev/null; then
    echo "❌ AWS CDK CLI not found. Install with: npm install -g aws-cdk"
    exit 1
fi

# Run tests
echo "🧪 Running tests..."
npm run test
npm run lint
npm run type-check

# Build application
echo "🔨 Building application..."
npm run build

# Deploy infrastructure
echo "🏗️ Deploying infrastructure..."

# Bootstrap CDK (if needed)
cdk bootstrap aws://$(aws sts get-caller-identity --query Account --output text --profile $PROFILE)/$REGION --profile $PROFILE

# Deploy stack
cdk deploy $STACK_NAME \
    --profile $PROFILE \
    --region $REGION \
    --parameters Environment=$ENVIRONMENT \
    --parameters DomainName=$DOMAIN_NAME \
    --require-approval never \
    --outputs-file outputs.json

# Post-deployment tasks
echo "📋 Running post-deployment tasks..."

# Extract outputs
API_URL=$(cat outputs.json | jq -r ".\"$STACK_NAME\".ApiUrl")
USER_POOL_ID=$(cat outputs.json | jq -r ".\"$STACK_NAME\".UserPoolId")
USER_POOL_CLIENT_ID=$(cat outputs.json | jq -r ".\"$STACK_NAME\".UserPoolClientId")

echo "✅ Deployment completed successfully!"
echo "📊 Deployment Summary:"
echo "  API URL: $API_URL"
echo "  User Pool ID: $USER_POOL_ID"
echo "  User Pool Client ID: $USER_POOL_CLIENT_ID"

# Update environment file
cat > .env.$ENVIRONMENT << EOF
VITE_API_BASE_URL=$API_URL
VITE_USER_POOL_ID=$USER_POOL_ID
VITE_USER_POOL_CLIENT_ID=$USER_POOL_CLIENT_ID
VITE_REGION=$REGION
VITE_ENVIRONMENT=$ENVIRONMENT
EOF

echo "📝 Environment file updated: .env.$ENVIRONMENT"

# Run smoke tests
echo "🔥 Running smoke tests..."
npm run test:smoke -- --env $ENVIRONMENT

echo "🎉 Deployment pipeline completed successfully!"
```

Infrastructure Validation Script (scripts/validate-infrastructure.js):
```javascript
const AWS = require('aws-sdk');
const fs = require('fs');

class InfrastructureValidator {
  constructor(region = 'us-east-1') {
    AWS.config.update({ region });
    this.cloudformation = new AWS.CloudFormation();
    this.apigateway = new AWS.APIGateway();
    this.lambda = new AWS.Lambda();
    this.dynamodb = new AWS.DynamoDB();
    this.cognito = new AWS.CognitoIdentityServiceProvider();
  }

  async validateStack(stackName) {
    console.log(`🔍 Validating stack: ${stackName}`);

    try {
      const stack = await this.cloudformation.describeStacks({
        StackName: stackName
      }).promise();

      const stackStatus = stack.Stacks[0].StackStatus;
      console.log(`✅ Stack Status: ${stackStatus}`);

      if (stackStatus !== 'CREATE_COMPLETE' && stackStatus !== 'UPDATE_COMPLETE') {
        throw new Error(`Stack is in invalid state: ${stackStatus}`);
      }

      return stack.Stacks[0];
    } catch (error) {
      console.error(`❌ Stack validation failed: ${error.message}`);
      throw error;
    }
  }

  async validateApiGateway(apiId) {
    console.log(`🔍 Validating API Gateway: ${apiId}`);

    try {
      const api = await this.apigateway.getRestApi({ restApiId: apiId }).promise();
      console.log(`✅ API Gateway: ${api.name}`);

      // Check deployment
      const deployments = await this.apigateway.getDeployments({ restApiId: apiId }).promise();
      if (deployments.items.length === 0) {
        throw new Error('No deployments found');
      }

      console.log(`✅ Deployments: ${deployments.items.length}`);
      return true;
    } catch (error) {
      console.error(`❌ API Gateway validation failed: ${error.message}`);
      throw error;
    }
  }

  async validateLambdaFunctions(functionNames) {
    console.log(`🔍 Validating Lambda functions: ${functionNames.join(', ')}`);

    for (const functionName of functionNames) {
      try {
        const func = await this.lambda.getFunction({ FunctionName: functionName }).promise();
        console.log(`✅ Lambda Function: ${func.Configuration.FunctionName} (${func.Configuration.State})`);

        if (func.Configuration.State !== 'Active') {
          throw new Error(`Function ${functionName} is not active`);
        }
      } catch (error) {
        console.error(`❌ Lambda validation failed for ${functionName}: ${error.message}`);
        throw error;
      }
    }
  }

  async validateDynamoDBTables(tableNames) {
    console.log(`🔍 Validating DynamoDB tables: ${tableNames.join(', ')}`);

    for (const tableName of tableNames) {
      try {
        const table = await this.dynamodb.describeTable({ TableName: tableName }).promise();
        console.log(`✅ DynamoDB Table: ${table.Table.TableName} (${table.Table.TableStatus})`);

        if (table.Table.TableStatus !== 'ACTIVE') {
          throw new Error(`Table ${tableName} is not active`);
        }
      } catch (error) {
        console.error(`❌ DynamoDB validation failed for ${tableName}: ${error.message}`);
        throw error;
      }
    }
  }

  async validateCognitoUserPool(userPoolId) {
    console.log(`🔍 Validating Cognito User Pool: ${userPoolId}`);

    try {
      const userPool = await this.cognito.describeUserPool({ UserPoolId: userPoolId }).promise();
      console.log(`✅ Cognito User Pool: ${userPool.UserPool.Name}`);

      // Check MFA configuration
      if (userPool.UserPool.MfaConfiguration !== 'OPTIONAL') {
        console.warn(`⚠️ MFA not configured as optional`);
      }

      return true;
    } catch (error) {
      console.error(`❌ Cognito validation failed: ${error.message}`);
      throw error;
    }
  }

  async runFullValidation(config) {
    console.log('🚀 Starting full infrastructure validation...\n');

    try {
      // Validate CloudFormation stack
      await this.validateStack(config.stackName);

      // Validate API Gateway
      await this.validateApiGateway(config.apiId);

      // Validate Lambda functions
      await this.validateLambdaFunctions(config.lambdaFunctions);

      // Validate DynamoDB tables
      await this.validateDynamoDBTables(config.dynamoTables);

      // Validate Cognito User Pool
      await this.validateCognitoUserPool(config.userPoolId);

      console.log('\n✅ All infrastructure validation checks passed!');
      return true;
    } catch (error) {
      console.error('\n❌ Infrastructure validation failed:', error.message);
      return false;
    }
  }
}

// Configuration
const config = {
  stackName: process.env.STACK_NAME || 'mycoolchatbot-dev',
  apiId: process.env.API_ID,
  lambdaFunctions: [
    'mycoolchatbot-auth-function',
    'mycoolchatbot-chat-function',
    'mycoolchatbot-user-function',
    'mycoolchatbot-ai-function'
  ],
  dynamoTables: [
    'mycoolchatbot-users',
    'mycoolchatbot-conversations',
    'mycoolchatbot-messages',
    'mycoolchatbot-audit-logs'
  ],
  userPoolId: process.env.USER_POOL_ID
};

// Run validation
const validator = new InfrastructureValidator();
validator.runFullValidation(config)
  .then(success => {
    process.exit(success ? 0 : 1);
  })
  .catch(error => {
    console.error('Validation error:', error);
    process.exit(1);
  });
```
~~~

Infrastructure Implementation Complete

Your Infrastructure phase implementation is now complete with:

✅ Implemented Components

- **Complete AWS infrastructure** with serverless architecture and auto-scaling
- **Comprehensive security controls** with WAF, GuardDuty, Security Hub, and encryption
- **Automated deployment pipeline** with validation, testing, and rollback capabilities
- **Operational excellence** with monitoring, alerting, and incident response

✅ Production-Ready Features

- **Serverless compute** with Lambda functions and API Gateway
- **Managed databases** with DynamoDB and ElastiCache for optimal performance
- **Authentication system** with Cognito User Pools and Identity Pools
- **Security monitoring** with comprehensive logging and threat detection
- **Cost optimization** with pay-per-use pricing and resource efficiency

🚀 Next Steps

With your infrastructure complete, you're ready to:

1. **Deploy the infrastructure** using the automated deployment pipeline
2. **Implement frontend components** that integrate with the API endpoints
3. **Build backend Lambda functions** that utilize the database and AI services
4. **Apply security controls** throughout the application layers

Your infrastructure provides a robust, scalable, and secure foundation for your production chatbot application.

Previous Next